Assignment Name: Project 1 Part 1

Class Name and Section: ECE452-002 **Date Submitted:** 03/15/2026

Group Members: Raghav Bharath (rsb49), Arnav Revankar (avr33), and Daniel Assaf (da67)

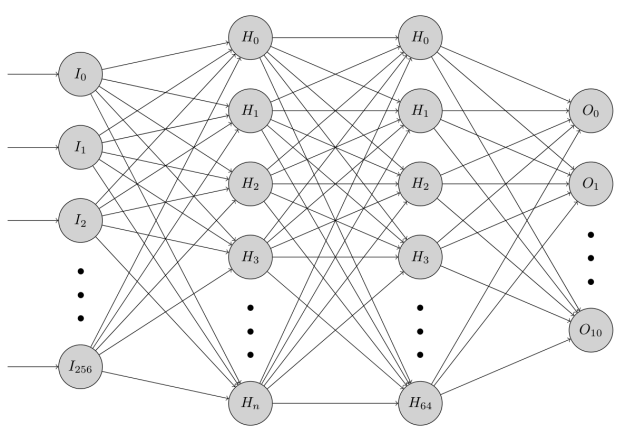
Introduction:

In this project, we architected, trained, and tested a custom feedforward multilayer perceptron (MLP) for handwritten digit recognition. For the initial training stage of the project, we used the PyTorch library to simplify the implementation of backpropagation-based learning. However, for the testing and inference portions of the project, we implemented a custom Neural Network class in Python, loaded the weights, and ran inference, using only the Numpy library for optimized matrix operations.

Once we had our baseline scripts, we used a second script to train and test *multiple networks* while varying hyperparameters to optimize performance. We ultimately succeeded and were able to reach 90% accuracy on the test dataset after training on the training dataset.

Procedure:

The structure of our network is shown below. The input, which was 256-dimensional (reflecting 16x16 pixel images), was directly mapped to *n* neurons in the first hidden layer and activated with ReLU. We made this value variable to test multiple configurations, and ultimately to extract the best-performing one. The output of the first hidden layer was then passed into a second hidden layer of 64 neurons, also activated with ReLU. Finally, the 64 neuron hidden layer is transformed into 10 output neurons, which are activated with the Softmax activation function. Unlike other activation functions, the softmax's outputs sum to 1, and are thus *dependent* on the values of other neurons. In a traditional Sigmoid, ReLU, or Tanh activation, each activation can be applied to one neuron at a time, which guarantees independence but does not guarantee solid predictions, especially for a one-hot encoded design.




```python
class MnistNetwork(nn.Module):
    def __init__(self, n: int = 128):
        super(MnistNetwork, self).__init__()
        self.Layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256, n),
            nn.ReLU(),
            nn.Linear(n, 64),
            nn.ReLU(),
            nn.Linear(64,10),
            #nn.Softmax(dim=1) - the softmax function here is not needed, as CrossEntropyLoss applies it internally
        )
```

The next step, testing, was done via a *completely custom* neural network model, only utilizing Numpy for matrix calculations, and Pytorch to initially load the weights from the .pt file. This involved writing the two classes shown below, as well as supporting functions for loading weights and inference.

Class for a single layer

```python
class Layer():
    # Layers are stored in the shape of 1xN vectors.
    # Matrix multiplication will follow xA^T
    def __init__(self, length, layernum, activation):
        self.PreActivation = np.empty(length)
        self.PostActivation = np.empty(length)
        self.Bias = np.empty(length)
        self.ActFunc = activation
        self.LayerNum = layernum

    def MatMul(self, weights, bias):
        ...

    def Activate(self):
        ...
```

Class for the entire network

```python
class Network():
    def __init__(self):
        self.layers = []
        self.weights = []

    def AddLayer(self, length, activation='ReLU'):
        ...

    def LoadParameters(self, PytorchFilePath):
        ...

    def Forward(self, Input: np.ndarray) -> np.ndarray:
        ...
```

Once we had the test script, we ran tests on several different networks, varying parameters such as neurons in the hidden layer, loss function, optimizer, and learning rate, to find the neural network that learned (but did not overfit!) the most within 2000 epochs on the 100 training images. Our results are documented below.

Results and Discussion:

To identify the optimal network configuration, we trained 32 models across all combinations of 4 hyperparameters: hidden neuron count (64, 128, 512, 1024), learning rate (0.001, 0.01), loss function (MSE and cross-entropy), and optimizer (SGD and Adam). We initially trained each model for 2,000 epochs on the 100-image training set and evaluated on the 50-image test set. Our plan was to choose the best performing model, and then see if we can get even better accuracy when we up the epochs to 2500. Some of our results are shown below. We found that some of the tests performed very poorly, reaching low accuracies like 14%.

Neurons	**LR**	**Loss**	**Optimizer**	**Test Accuracy**
1024	0.01	CE	Adam	88%
1024	0.01	MSE	Adam	88%
512	0.01	CE	SGD	86%
128	0.01	CE	SGD	82%
64	0.01	CE	Adam	70%
1024	0.001	CE	Adam	14%

We noticed a clear trend: the learning rate had probably the largest impact on the performance. The models that were trained with 0.001 all performed relatively poorly, having accuracies of around 8-30%. However, the models with learning rates of 0.01 achieved 70-88%, which is quite impressive. This is likely

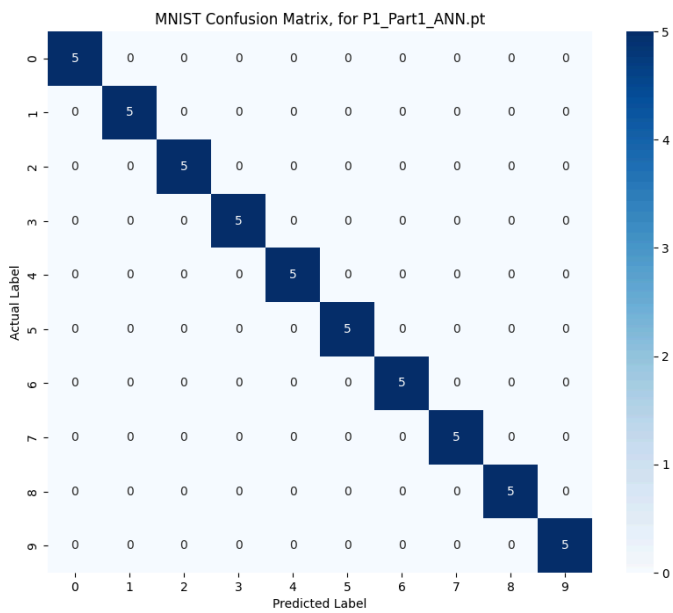
because with a learning rate of 0.001, the model struggles to converge within 2,000 epochs, given the network capacity.

Amongst the lr = 0.01 models, the larger neuron counts (512, 1024) typically outperformed the smaller ones, which suggests that the network benefited from increased capacity even with only 100 training samples. The Adam optimizer also showed marginal advantages over SGD in most configurations. Lastly, the cross-entropy loss generally did better than the MSE for the classification tasks, which is consistent because it is theoretically supposed to produce stronger gradients and lead to faster training.

Looking specifically at the effect of neuron count, increasing from 64 to 1024 neurons at lr=0.01 generally improved test accuracy from 70% to 88%. However, at lr = 0.001, neuron count had almost no effect since the learning rate was too small to converge regardless of capacity. More neurons help, but only when paired with a learning rate that actually allows the network to learn.

Another thing we noticed is that, looking at the loss curves for all 2,000 epoch runs, the convergence doesn't seem to be fully reached. The loss seems to continue to decrease even as we get closer to 2,000 epochs. So, we decided to try out the best performing model out of the 2,000 epoch lot, which is the 1024 neurons, lr=0.01, CE, Adam model, with 2,500 epochs.

Interestingly (and extremely luckily), this retraining run converged to near-zero loss by around epoch 50, with no meaningful change for the remaining 2,450 epochs. We think this came down to random weight initialization. Since PyTorch randomly initializes weights at the start of each run, the 2,500-epoch run likely started from a more favorable position, allowing it to converge much faster than the original run. This model achieved 100% accuracy on both the 50-image test set and the 100-image training set, as shown in the confusion matrix below. We initially suspected overfitting since the training loss hit near-zero, but the 100% test accuracy suggests the model generalized well to all provided test samples.



Note: We mislabeled the confusion matrix title as "MNIST Confusion Matrix,"
but the data shown reflects the custom-provided digit dataset, not the standard MNIST dataset.

Conclusion:

In this project, we designed, trained, and evaluated a custom MLP for handwritten digit recognition. After sweeping across 32 hyperparameter configurations, we found that the learning rate was the most important factor affecting convergence, with lr = 0.01 consistently outperforming lr = 0.001 across all neuron counts, loss functions, and optimizers. We also noticed that increasing the hidden neuron count generally improved

accuracy, with 1024 neurons at lr=0.01 giving us the best results. Our optimal model, which used 1024 neurons, lr=0.01, cross-entropy loss, and the Adam optimizer trained for 2,500 epochs, achieved 100% test accuracy on the provided 50-image test set. We also successfully implemented a custom NumPy-based inference engine that replicated PyTorch's forward pass at the individual neuron level without using any built-in framework inference functions. Going forward, we think accuracy could be further improved through data augmentation to address the limited training set size, dropout or L2 regularization to reduce overfitting risk, and feature extraction techniques like HOG descriptors to give the network more discriminative inputs.

References:

1. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980
2. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929-1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html
3. MathWorks, "Pattern Recognition with a Shallow Neural Network," MathWorks Documentation, 2024. [Online]. Available: https://www.mathworks.com/help/deeplearning/gs/pattern-recognition-with-a-shallow-neural-network.html
4. Pytorch Contributors, Python API Reference, 2025. [Online]. Available: https://docs.pytorch.org/docs/stable/pytorch-api.html
5. TensorFlow Contributors, "Multilayer Perceptrons," TensorFlow Documentation, 2024. [Online]. Available: https://www.tensorflow.org/guide/core/mlp_core